

Algorithm Lab

Assignment 7: Dijkstra's Algorithm

Dijkstra's algorithm is a one-to-all shortest path algorithm. Most of shortest path algorithms work on directed weighted graph. Dijkstra's algorithm not always work correctly on an arbitrary graph, it can only apply on graphs without negative cycle. We can define directed weighted graph by $G = (V, E)$ where V is its vertices set and E is its edges set. And we can denote an edge by its 2 ends and its weight, in mathematical description, $e_i = (s_i, t_i, w_i) \in E$, $s_i, t_i \in V$.

A s -to- t path is a sequence of edges $P = (e_1, e_2, \dots, e_n)$ that $\forall 1 \leq i < n, t_i = s_{i+1}$ and $s_1 = s, t_n = t$. The cost of P is $\sum_{e_i \in P} w_i$. Assume that we have 2 paths P_a and P_b , we say P_a shorter than P_b if cost of P_a less than cost of P_b . Respectively, the shortest path of a starttarget pair is the path with minimal cost. One-to-all shortest path (or Single Source Shortest Path, SSSP) problem, by its name, is ask to find shortest paths with a given start vertex. Defined as followed:

Instance: weighted graph $G = (V, E)$ and start vertex $s \in V$.

Result: Some data structure can query shortest $s - t$ path by a given t .

Description

Dijkstra's algorithm is to find and to store shortest paths in a kind of rooted spanning tree.

We can denote the result by its edges set $T \subseteq E$. In Dijkstra's algorithm, we divide vertices to 2 groups, visited group V' and unvisited group U .

If we want to move a vertex $v_i \in U$ to V' , we should apply our operation by a bridge edge $e_b = (s_b, v_i, w_b) \in E$ and $s_b \in V'$.

1. $V' \leftarrow \{s\}, U \leftarrow V \setminus V', T \leftarrow \{(s, s, 0)\}$.
2. Find the minimum bridge edge e_b between V' and U .
3. Find bridged node $e_r \in T$ that $t_r = s_b$ with minimal $w_b + w_r$.
4. Make an edge $e'_b = (s_b, t_b, w_b + w_r)$.
5. Update information $V' \leftarrow V' \cup \{t_b\}, U = U \setminus \{t_b\}, T \leftarrow T \cup \{e'_b\}$.

6. Repeat step 2 to step 5 until $U = \emptyset$.

Questions

- 1 Analyze the time complexity when implement step 2 by sequential searching whole adjacency list.
- 2 Analyze the time complexity when implement step 2 by a priority queue that maintained the bridge edges.
 - 2.1 Implement the priority queue by double-linked list.
 - 2.2 Implement the priority queue by Min-heap.
- 3 Design an algorithm to reconstruct the path from spanning tree T (the result).
- 4 How to find the unreachable (have no path from s) set?
- 5 Solve <http://oj.csie.ndhu.edu.tw/problem/ALG07C> and analyze your code.